Design, Simulation and Testing of a 5-Stage Pipelined RISC-V Processor Using Verilog HDL

By: Muhammad Arham Bsce24054

Laraib Noor Bsce24018

1. Introduction

RISC-V is an open-source Instruction Set Architecture (ISA) widely used for processor design and research. In this project, an existing single-cycle RISC-V processor was modified into a 5-stage pipelined processor to improve instruction throughput and overall performance.

The processor executes multiple instructions simultaneously by dividing instruction execution into five stages. Pipeline registers were inserted between stages to allow concurrent execution. Data hazards were handled using forwarding and stalling mechanisms.

2. Pipelined Datapath Design

The single-cycle datapath was converted into a pipelined datapath consisting of five stages:

1. Instruction Fetch (IF)
2. Instruction Decode (ID)
3. Execute (EX)
4. Memory Access (MEM)
5. Write Back (WB)

Pipeline registers were inserted between adjacent stages:

- IF/ID Register
- ID/EX Register
- EX/MEM Register
- MEM/WB Register

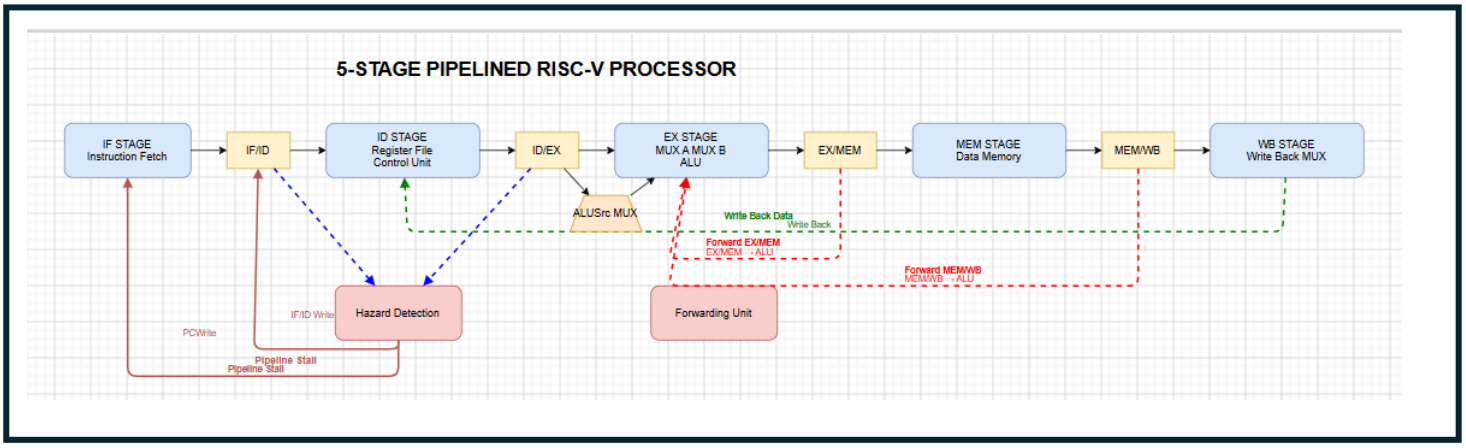
These registers store intermediate data and control signals between clock cycles.

Block Diagram

The pipelined processor contains:

- Program Counter (PC)
- Instruction Memory
- Register File
- Control Unit
- ALU
- Data Memory
- Write Back Multiplexer
- Forwarding Unit
- Hazard Detection Unit
- IF/ID, ID/EX, EX/MEM and MEM/WB Pipeline Registers

The forwarding unit provides operand forwarding to the ALU, while the hazard detection unit stalls the pipeline when necessary.



3. Description of Pipeline Stages

3.1 Instruction Fetch (IF)

The IF stage fetches instructions from instruction memory using the Program Counter (PC). The PC is incremented by 4 after each instruction fetch.

Output:

- Instruction
- PC + 4

Stored in:

- IF/ID Register

3.2 Instruction Decode (ID)

The ID stage decodes the fetched instruction and reads source operands from the register file. The control unit generates control signals required by later stages.

Operations:

- Register read
- Instruction decoding
- Control signal generation

Stored in:

- ID/EX Register

3.3 Execute (EX)

The EX stage performs arithmetic and logical operations using the ALU.

Operations:

- Addition
- Subtraction
- Address calculation
- Comparison operations

The forwarding unit forwards data from later pipeline stages when dependencies occur.

Stored in**: EX/MEM Register**

3.4 Memory Access (MEM)

The MEM stage accesses data memory for load and store instructions.

Operations:

- Read data memory
- Write data memory

Stored in:

- MEM/WB Register

3.5 Write Back (WB)

The WB stage writes ALU results or memory data back to the destination register in the register file.

Operations:

- Register update
- Result selection through Write Back MUX

4. Hazard Handling Mechanism

Data Hazard Handling

Data hazards occur when an instruction depends on the result of a previous instruction that has not completed execution.

Example:

add x3, x1, x2

sub x4, x3, x5

The second instruction requires x3 before it has been written back.

Forwarding

A forwarding unit was implemented to reduce stalls.

Forwarding paths:

- EX/MEM → EX Stage
- MEM/WB → EX Stage

The forwarding unit compares source and destination register addresses and selects forwarded data whenever a dependency is detected.

Stalling

For load-use hazards, forwarding alone is insufficient.

Example:

lw x5, 0(x1)

add x6, x5, x2

The hazard detection unit detects this condition and inserts a stall cycle.

The stall mechanism:

- Freezes Program Counter
- Freezes IF/ID register
- Inserts a bubble into the pipeline

5. Verilog Modifications

The following major changes were introduced:

Pipeline Registers

Separate pipeline registers were added:

- IF/ID
- ID/EX
- EX/MEM
- MEM/WB

These registers store instruction data and control signals between stages.

Forwarding Unit

A forwarding unit was added to detect register dependencies and select forwarded ALU operands.

Outputs:

- ForwardA
- ForwardB

Hazard Detection Unit

A hazard detection unit was implemented to detect load-use hazards.

Outputs:

- Stall
- PCWrite
- IF/ID Write

6. Simulation and Testing

Test Program

The processor was tested using a sequence of instructions containing:

- ADD
- SUB
- ADDI
- LW
- SW

The test program included dependent instructions to verify forwarding and stalling functionality.

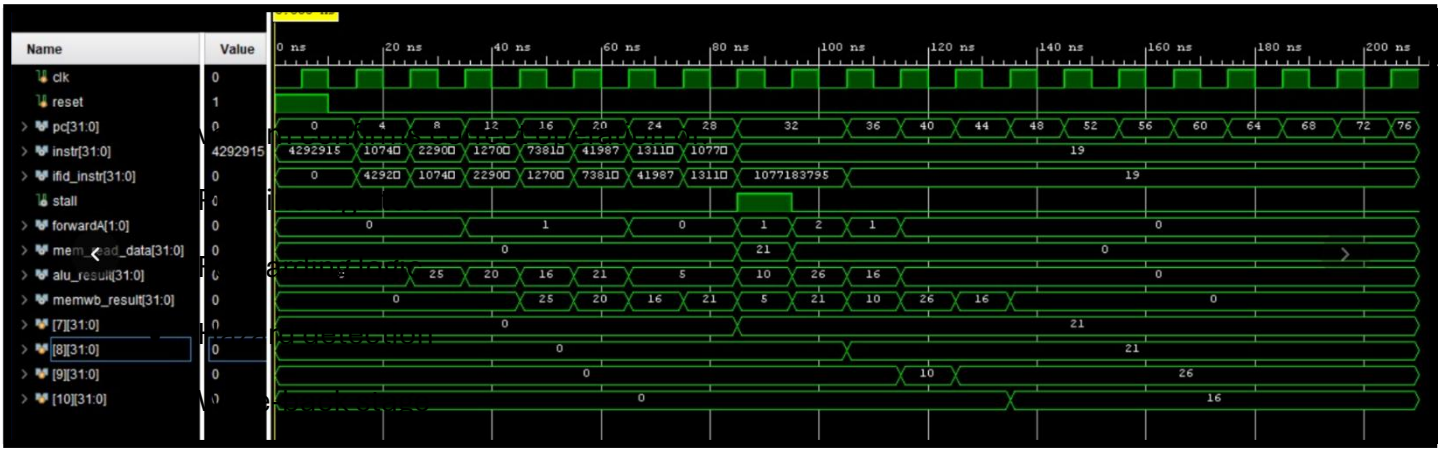
7. Simulation Results

Simulation was performed using Verilog HDL and waveform analysis.

Observations:

1. The Program Counter increased correctly by 4 bytes every cycle.
2. Instructions progressed through IF, ID, EX, MEM and WB stages.
3. Forwarding signals became active when data dependencies occurred.
4. The stall signal was asserted during load-use hazards.

5. ALU results propagated correctly through pipeline registers.
6. MEM/WB results were written back successfully to the register file.
7. Register values matched expected results after execution.



8. Analysis

Compared to the original single-cycle processor, the pipelined design allows multiple instructions to execute simultaneously.

Advantages observed:

- Higher instruction throughput
- Reduced execution time
- Improved processor utilization

The forwarding unit significantly reduced the number of stall cycles, while the hazard detection unit ensured correct program execution.

9. Conclusion

A 5-stage pipelined RISC-V processor was successfully designed and simulated using Verilog HDL. Pipeline registers were introduced between stages, and data hazards were handled using forwarding and stalling techniques. Simulation results verified correct instruction execution and proper operation of hazard management mechanisms. The project demonstrated the performance benefits of pipelining and provided practical experience in modern processor design.